

```
a=10
b=10.5
c="Hello"
d=1j
e=["apple", "banana", "mango"]
f=("apple", "banana", "mango")
g=range(5)
```

```
print("Type of a:", a, type(a))
print("Type of b:", b, type(b))
print("Type of c:", c, type(c))
print("Type of d:", d, type(d))
print("Type of e:", e, type(e))
print("Type of f:", f, type(f))
print("Type of g:", g, type(g))
```

```
Type of a: 10 <class 'int'>
Type of b: 10.5 <class 'float'>
Type of c: Hello <class 'str'>
Type of d: 1j <class 'complex'>
Type of e: ['apple', 'banana', 'mango'] <class 'list'>
Type of f: ('apple', 'banana', 'mango') <class 'tuple'>
Type of g: range(0, 5) <class 'range'>
```

```
# Activity 1
print("Hello World")
# this is a comment
num1=5
num2=4
if num1==num2:
    print("Equal")
else:
    print("Not equal")

print("Enter your name:")
name=input()
print("Name is :", name)

print("This is how we print a statement")
```

```
Hello World
Not equal
Enter your name:
ayesha
Name is : ayesha
This is how we print a statement
```

```
# LAB TASKS
# Task 1
# a=10 : int
# b=10.0 : float
# c="10" : str
# d=True : bool
# e=3+4j : complex
```

```
a=10
b=10.0
c="10"
d=True
e=3+4j
print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'complex'>
```

```
# Task 2
x=input("Enter 1st number:")
y=input("Enter 2nd number:")
print("Type before conversion:")
print(type(x))
print(type(y))

f1=float(x)
f2=float(y)
print("Type after conversion:")
print(type(f1))
print(type(f2))

sum = f1 + f2
print("Sum is: {}".format(sum))
```

```
Enter 1st number:2
Enter 2nd number:3
Type before conversion:
<class 'str'>
<class 'str'>
Type after conversion:
<class 'float'>
<class 'float'>
Sum is: 5.0
```

```
#task 3
name = input("Enter name:")
age = int(input("Enter age:"))
cgpa = float(input("Enter CGPA:"))

print("Student Name : {}".format(name))
print("Age          : {} years".format(age))
print("CGPA         : {}".format(cgpa))
```

```
Enter name:ayesha
Enter age:22
Enter CGPA:3.94
Student Name : ayesha
Age          : 22 years
CGPA         : 3.94
```

```
#task 4
a = input("write something: ")

if "." in a:
    a= float(a)
else:
    a= int(a)

print(a)
print(type(a))
```

```
write something: 4.0
4.0
<class 'float'>
```

```
#task 5
n1=int(input("Enter 1st number:"))
n2=int(input("Enter 2nd number:"))
print(n1)
print(n2)

print(n1>0 and n2>0)
print(n1==0 or n2==0)
print(n1==n2)
```

```
Enter 1st number:7
Enter 2nd number:0
7
0
False
```

```
True
False
```

```
# task 6
"""
this is the first line of the comment.
i am trying to write a multiline comment
"""
print("Multiline comments are not executed")
```

```
Multiline comments are not executed
```

```
#task 7
#source code written by programmer
x=10
y=10
sum=x+y

# Byte Code

# The Python interpreter compiles the source code into bytecode.
# This bytecode is not visible to the programmer.

# PVM Execution

# The Python Virtual Machine (PVM) executes the bytecode line by line.
# This is why Python is called an interpreted language.

# Final Output
print("Python is compiled to bytecode and then interpreted by PVM")
```

```
Python is compiled to bytecode and then interpreted by PVM
```

```
#task 8
x=10
print(x, type(x))
x="Ten"
print(x, type(x))
x=10.5
print(x, type(x))
```

```
10 <class 'int'>
Ten <class 'str'>
10.5 <class 'float'>
```

```
# task 9
#task9
num = int(input("Enter a number: "))

if num > 0:
    print("Positive Number")
elif num<0:
    print("Negative Number")
else:
    print("Zero")
```

```
Enter a number: 0
Zero
```

```
#task 10
num1 = 10
num2 = 10.5
num3 = "ayesha"
num4 = True
num5 = [85, 90]

print("Value: {} | Type: {}".format(num1, type(num1)))
print("Value: {} | Type: {}".format(num2, type(num2)))
print("Value: {} | Type: {}".format(num3, type(num3)))
```

```
print("Value: {} | Type: {}".format(num4, type(num4)))  
print("Value: {} | Type: {}".format(num5, type(num5)))
```

```
Value: 10 | Type: <class 'int'>  
Value: 10.5 | Type: <class 'float'>  
Value: ayesha | Type: <class 'str'>  
Value: True | Type: <class 'bool'>  
Value: [85, 90] | Type: <class 'list'>
```